

Prediction Algorithm Accuracy Report — v5

A Minutes-Trend Feature That Actually Worked

Sports Analytics Project

September 2026

Abstract

Prior reports (v2, v3) rejected two attempts at a minutes-aware player-point predictor: a multiply-based approach (predicted minutes $\times$ predicted per-minute rate, replacing the existing prediction) that compounded two noisy estimates and made RMSE worse in both models it was tried on. This report tests a third, structurally different idea — a small *adjustment* layered on top of the existing recency-weighted prediction, driven by whether a player’s recent minutes have trended away from their own longer-run baseline, rather than replacing the prediction with a minutes-derived one. Backtested with the same discipline as every prior experiment in this series (chronological train/validation split, cross-checked on two independent splits, a targeted check on players with an actual minutes swing), this one is a real, consistent win for the fantasy-points predictor (`apps/optimizer/predict.py`) and a genuine null result for the scorer-points predictor (`game-detail.service.ts`). Shipped to the former only.

Contents

1	Why a Third Attempt at Minutes	1
2	Method	2
2.1	The adjustment	2
2.2	Backtest	2
2.3	Targeted check: does the effect concentrate where it should?	2
3	Why the Two Models Diverge	3
4	What Shipped	3
5	Recommendations Going Forward	3

1 Why a Third Attempt at Minutes

Two of the three ideas named as “what’s next” in earlier reports (opponent-adjustment, a multiply-based minutes model) were tried and rejected — see v2 and v3’s module-comment write-ups. Both failures shared a root cause: they replaced an existing single-source-of-noise prediction with a derived quantity built from two independently noisy estimates, which increases variance even when it nudges the mean estimate in the right direction. This report deliberately avoids that failure mode: the existing recency-weighted prediction is kept as the baseline in full, and only a small, capped, damped correction is layered on top of it.

2 Method

2.1 The adjustment

For each prediction, a *trend ratio* is computed: the mean of a player’s most recent 3 games’ minutes, divided by the mean of their most recent 10 games’ minutes. A ratio above 1 means recent minutes exceed the longer-run baseline (a role expansion); below 1 means the opposite (a role reduction, injury return still ramping up, etc.). The existing recency-weighted prediction is then scaled by $1 + \text{strength} \times \text{clamp}(\text{ratio} - 1, -1, 1)$ — a damped, capped nudge, not a replacement. A player with fewer than 10 games of history gets no adjustment (trend is undefined without enough baseline).

2.2 Backtest

The same chronological train/validation methodology as every prior experiment: a grid over the adjustment strength was searched on a training split, the winner was checked against that split’s held-out validation portion, and the whole process was repeated on a second, independent split (70/30 and 60/40) to confirm the winner wasn’t specific to one split’s particular cut point.

Model	Split	Strength (train-winner)	Validation edge (MAE)
Fantasy points	70/30	0.15	+0.0149
Fantasy points	60/40	0.15	+0.0208
Scorer points	70/30	0.05	+0.0010
Scorer points	60/40	0.00	+0.0000

Table 1: Grid-search winner and validation-split edge (positive = adjustment helps), both models, both splits. Fantasy points shows the same winning strength on both splits with a consistent, non-trivial edge. Scorer points shows a negligible-to-zero edge, and its 60/40 split’s own grid search picked strength=0.0 — no adjustment — as optimal.

2.3 Targeted check: does the effect concentrate where it should?

If the mechanism is real, it should help most specifically on players whose minutes actually changed recently — not uniformly across the whole population, where most players’ minutes are stable and there’s nothing for a trend signal to catch.

Model	Split	Edge on players with $ \text{trend ratio} - 1 > 0.3$
Fantasy points	70/30	+0.0672
Fantasy points	60/40	+0.0796
Scorer points	70/30	+0.0063
Scorer points	60/40	+0.0000

Table 2: Edge restricted to the subset of predictions where the player had a genuine recent minutes swing. Fantasy points’ edge is 4-5 $\times$ larger on this subset than on the full population — exactly the concentration pattern a real minutes-driven effect should show. Scorer points shows no such concentration.

This is a meaningfully stronger form of evidence than the aggregate MAE numbers alone: fantasy points’ edge growing substantially larger specifically on the population the adjustment targets, on

both independent splits, is hard to explain as noise. Scorer points showing no concentration at all — not even a modest one — is equally informative in the other direction.

3 Why the Two Models Diverge

`game-detail.service.ts`'s scorer-points predictor already caps its recency window at `MOST_RECENT_GAMES_CONSIDERED = 10` games, versus `predict.py`'s fantasy-points predictor, which uses an unbounded window (every game in a player's history, geometrically down-weighted). A capped 10-game window is already substantially more locally responsive to a recent role change than an unbounded one — the oldest, most stale games simply fall out of the window entirely rather than being merely down-weighted. This is a plausible, mechanistic explanation for why a separate minutes-trend signal has room to add value in one predictor and not the other, rather than the two predictors' underlying reality actually differing.

4 What Shipped

`apps/optimizer/predict.py` now computes a minutes-trend ratio per player (3-game recent window vs. 10-game baseline window) and applies a damped, capped adjustment (strength 0.15, deviation capped at ± 1.0) on top of the existing recency-weighted fantasy-point prediction. `PlayerPrediction` was regenerated for all players under the new model. `game-detail.service.ts` was left unchanged, with the null result documented in its module comment alongside the two previously-rejected ideas.

11 new unit tests (`test_predict.py`, in a newly-created `apps/optimizer` test suite — this app had no tests before this change) cover the trend-ratio calculation's edge cases (insufficient history, zero baseline minutes) and the adjustment function's behavior (no-op at ratio 1.0 or `None`, correct direction and magnitude, clamping at the deviation cap). All pass, alongside the full existing 187-test suite across the predictor, API, and web apps (198 total after this addition), with no regressions.

5 Recommendations Going Forward

1. **This is now the third consecutive experiment that isolated one variable, checked it two independent ways, and shipped only what survived both** (multi-season Elo tuning in v3, the season-boundary reset in v4, this minutes-trend adjustment in v5) — alongside three rejected ideas in the same period (opponent-adjustment, the multiply-based minutes model, and this same trend adjustment on the scorer-points predictor specifically). The hit rate is roughly even, which is the expected and healthy outcome of actually testing ideas rather than shipping on intuition.
2. **Four Factors remains the least-improved model across this entire report series** and has had no isolated, backtested experiment run against it directly (only the trade-attribution fix and the passive benefit of more data). It is the most under-explored remaining target — a natural next step given the other three models have each now had at least one dedicated, validated improvement.
3. **The scorer-points predictor's capped 10-game window** being the plausible reason it didn't benefit from this adjustment is itself testable, not just a hypothesis — a follow-up

could check whether widening or removing that cap changes the minutes-trend result, though that would need its own isolated backtest against the window-size question first.

Appendix: Reproducing These Results

The grid-search and validation-split backtest script behind this report is one-off analysis, not part of the committed codebase, matching the same appendix note in v3 and v4. `test_predict.py` in `apps/optimizer` exercises the shipped adjustment's exact logic (not an approximation of it) and is the durable, re-runnable check that the shipped behavior matches what was validated here.